

Laboratório de Estrutura de Dados

Primeira versão do projeto da disciplina

Comparação entre os algoritmos de ordenação elementar

Identificação do aluno,

Samuel Alves Correia - Matrícula: 191080098

1. Introdução

Este relatório visa apresentar os resultados e desempenho obtidos por cada algoritmo estudado na disciplina de LEDA. Para a implementação dos algoritmos, foi utilizada a linguagem Java, bem como um dataset de senhas, que foi disponibilizado pelo professor para análise.

Foram realizadas transformações no dataset disposto, que incluem a conversão do formato de datas para DD/MM/AAAA e a filtragem de senhas para as categorias Boa e Muito Boa. Além disso, também foi necessário fazer ordenações, a partir do arquivo *passwords_formated_data.csv*, gerado anteriormente pelas transformações. Dentre as ordenações pedidas, incluem: ordenação pelo campo length em ordem decrescente, ordenar o arquivo de senhas por mês e ordenar o arquivo completo de senhas de forma crescente.

2. Descrição geral sobre o método utilizado

Para o projeto foi utilizado o Eclipse e a linguagem de programação definida foi Java. Foram utilizadas as interfaces Comparable e Comparator como uma forma de definir ordens diferentes para os objetos. Este método se mostrou eficiente para as ordenações solicitadas no projeto.

Descrição geral do ambiente de testes

Para desenvolvimento e testagem dos algoritmos, foi utilizado um laptop, com processador Intel Core i7-8565U @ 1.8GHz, memória ram de 8GB, sistema operacional Windows 11, com arquitetura x64.

3. Resultados e Análise

Tabela de tempo (em milissegundos) da ordenação por data (DD/MM/AAAA)

Algoritmo	Melhor Caso	Médio Caso	Pior Caso
Counting Sort	2345	2907	4071
Heap Sort	375	953	484
Insertion Sort	6805	9602557	534867
Quick Sort	11068	10075	10075
Mediana de 3	2066	2889	2037
Selection Sort	4023898	5480821	2717290
Merge Sort	1027444	1060890	993479
3-Way QuickSort	281	522	221

Tabela de tempo (em milissegundos) da ordenação por mês

Algoritmo	Melhor Caso	Médio Caso	Pior Caso
Counting Sort	4061	5524	7260
Heap Sort	618	703	713
Insertion Sort	19	8214797	4657922
Quick Sort	StackOverflow	StackOverflow	StackOverflow
Mediana de 3	2447	109762	69618

Selection Sort	3897070	2560833	2364797
Merge Sort	973297	1067971	1067971
3-Way QuickSort	141	248	159

Tabela de tempo (em milissegundos) da ordenação por length

Algoritmo	Melhor Caso	Médio Caso	Pior Caso
Counting Sort	218	47	89
Heap Sort	781	354	1757
Insertion Sort	23	4034964	6010589
Quick Sort	StackOverFlow	StackOverFlow	StackOverFlow
Mediana de 3	StackOverFlow	StackOverFlow	StackOverFlow
Selection Sort	4023898	5480821	2717290
Merge Sort	35647462	2153120	2118709
3-Way QuickSort	353	85	74

Obs: Durante os testes houve um stack overflow na ordenação dos quickSorts, desta forma, optei por utilizar uma versão alternativa do algoritmo (3-way QuickSort), que consiste em dividir o array em: menor que o pivô, igual ao pivô e maiores que o pivô. O algoritmo se mostrou eficiente e demonstrou os resultados esperados.

Análise dos Algoritmos e Resultados

Os algoritmos de melhor desempenho, foram Counting Sort, HeapSort e 3-Way QuickSort com os melhores tempos perante a análise de tempo de complexidade dos algoritmos. Os algoritmos de pior desempenho foram Insertion Sort, Selection Sort e Merge Sort. Quick Sort não teve o desempenho esperado inicialmente, pela quantidade de valores repetidos do arquivo, entretanto, teve um bom desempenho na ordenação por DD/MM/AAAA. Entretanto, como citado anteriormente, ele obteve um stack overflow quando feita a ordenação por mês

Dentre os algoritmos que apresentaram melhor desempenho no melhor caso, enquadra-se o insertion sort, destacando-se que ele recebe um arquivo de entrada ordenado, tendo todos na posição correta e, sendo assim, cada elemento tem custo $O(1)$. O custo total é $O(n)$, pois a ordenação é executada n vezes. Apesar deste algoritmo ter obtido um bom desempenho no melhor caso, para o nosso estudo, ele não é o mais indicado, pois é ineficiente em entradas muito grandes. Neste caso, os algoritmos Counting Sort e Heap Sort se apresentam como uma melhor alternativa.

Counting Sort e Heap Sort apresentaram os melhores resultados em relação aos tempos de execução de melhor, médio e pior caso. O heap teve melhor desempenho que o Counting quando comparado por formato DD/MM/AAAA, já o Counting apresentou um melhor desempenho que o Heap quando ordenado por mês.

Desta forma, é possível concluir que dentre todos os algoritmos estudados, analisados e testados, o que se saiu melhor de acordo com o projeto elaborado, foi o Counting Sort.